

miHMI — HMI de captación de pozos

HMI minimalista sobre **Cheap Yellow Display** (ESP32-2432S028R, ILI9341 320×240 + XPT2046, LVGL 8.4). Muestra **2 estaciones de bombeo** y publica/monitorea sus variables. Es **cliente Modbus TCP** del **MAPA B** del contrato

`../ORCHESTRATION/REGISTER_MAP.md` — habla con el **LOGO! 9** real o con el **PLC-SIM** (`../modbusMaster`).

Arquitectura de datos

```
LOGO! 9 / PLC-SIM —(Modbus TCP :502, MAPA B)→ miHMI
```

- `src/data/data_source.h` — interfaz **DataSource** (begin/poll/sendCommand/health).
- `src/data/data_hub.{h,cpp}` — fuente activa con **failover** primaria→respaldo y cola de comandos.
- `src/data/modbus_tcp_source.{h,cpp}` — **primaria**: WiFi STA + cliente Modbus TCP asíncrono (`emelianov/modbus-esp8266`). Lee HR `s*32+0..13` por estación y el bloque global `IR 2000..2009`.
- `src/data/mock_source.{h,cpp}` — **respaldo / desarrollo sin PLC**.
- `src/data/map_b.h` — offsets del MAPA B (espejo del contrato).

`main.cpp`: `setPrimary(&g_plc); setBackup(&g_mock)`. Si la WiFi o el PLC no responden, el HUD cae al simulado automáticamente.

Configuración (`include/config.h`)

Símbolo	Uso
<code>WIFI_SSID / WIFI_PASS</code>	red de planta; vacío = no conecta y queda el respaldo
<code>PLC_HOST / PLC_PORT / PLC_UNIT</code>	destino Modbus TCP (IP del LOGO! o del PLC-SIM; <code>502</code> ; Unit ID <code>1</code>)
<code>MB_POLL_MS</code>	periodo de sondeo del MAPA B (750 ms)
<code>NUM_WELLS</code>	2 estaciones
<code>MAP_B_WORD_HI_FIRST</code>	orden de palabra de los 32 bits (contrato §2)

Pantallas

- **Estaciones** (`screen_wells`): 2 tarjetas con margen superior (cabecera de 46 px con origen SIM/LOGO + latido + alarma global). Cada tarjeta: nivel, caudal, presión/sirena/tapa, RSSI, estado (OK / ALARMA / SIN ENLACE).
- **Detalle** (`screen_well`): arco de nivel, caudal, acumulado del día, primera alarma activa; botones **SILENCIAR** y **SIRENA AUTO/MAN**.

- **Rangos de escala** (`screen_scale`, desde *Ajustes*): edita el bloque `hb+20..31` del MAPA B por estación y variable (cero/span crudo ↔ ingeniería, unidad, filtro) con teclado numérico, y lo **APLICA** al PLC (escribe los registros + pulsa el coil `cb+8`). La escala vive en el PLC; esto es el editor.

`CmdType` = superficie real del MAPA B: `SirenOn/Off`, `SirenAuto/Manual`, `Silence`, `ResetDay`, `ResetMonth`. `WellData` refleja el MAPA B (presostato, voltaje local, tamper, sirena, `alarms` bitfield, enlace, RSSI...).

Pendiente: mover `WIFI_SSID` / `PLC_HOST` a una pantalla de ajustes + NVS (hoy son `#define` en `config.h`).

Compilar

```
pio run -e esp32-2432S028R
pio run -e esp32-2432S028R -t upload
```

Probar sin PLC real

1. Levanta el **PLC-SIM**: en `../modbusMaster`, `python app.py`, pestaña *PLC-SIM*, apunta al gateway (`nodeIO_master`), inicia. Anota su IP:puerto.
2. Pon esa IP en `PLC_HOST` (y `502`, o el puerto que uses) y tu WiFi en `WIFI_SSID/WIFI_PASS`. Recompila y flashea.
3. En *Ajustes* / *Diagnóstico* la fuente activa debe pasar a **PLC-TCP**.